Сума ряду Тейлора за допомогою функцій

Звіт • Джамалов Олександр Олегович • PI-11

Обчислити і вивести на екран у вигляді таблиці значення функції, заданої за допомогою ряду Тейлора, на інтервалі від $x_{поч}$ до $x_{кін}$ з кроком dx та точністю ε. Параметри **xпоч**, **xкін**, **dx**, **eps** вводяться з клавіатури. Таблиця має містити заголовок та шапку. Кожний рядок таблиці має містити значення аргументу, значення функції, значення суми ряду та кількість порахованих доданків. Суму ряду Тейлора та значення доданку обчислювати за допомогою допоміжних алгоритмів, реалізованих за допомогою окремих функцій. При обчисленні значення доданків використовувати рекурентні співвідношення. Всю необхідну функціям інформацію слід передавати лише за допомогою параметрів.

Варіант 7.

$$\operatorname{arcctg} x = \frac{\pi}{2} + \sum_{n=0}^{\infty} \frac{(-1)^{n+1} x^{2n+1}}{2n+1} = \frac{\pi}{2} - x + \frac{x^3}{3} - \frac{x^5}{5} - \ldots, \quad |x| <= 1.$$

a_0 = -x; a_i = -a_(i-1)*x²*(2i-1)/(2i+1), i ≥ 1.
S = pi/2 + сума a_i. Доданок і сума обчислюються окремими функціями.
Зупинка: |останній доданок| < eps. Для |x| ≤ 1 ряд знакозмінний,
модулі доданків спадають, тому похибка відсікання менша за eps.
Кількість доданків включає a_0, але не константу pi/2.
Еталон: arccot(x) = pi/2 - atan(x), значення в інтервалі (0, pi).
Обмеження запуску: до 1 000 001 рядка й 10 000 000 доданків.

Структура: main → taylorSum → nextTerm

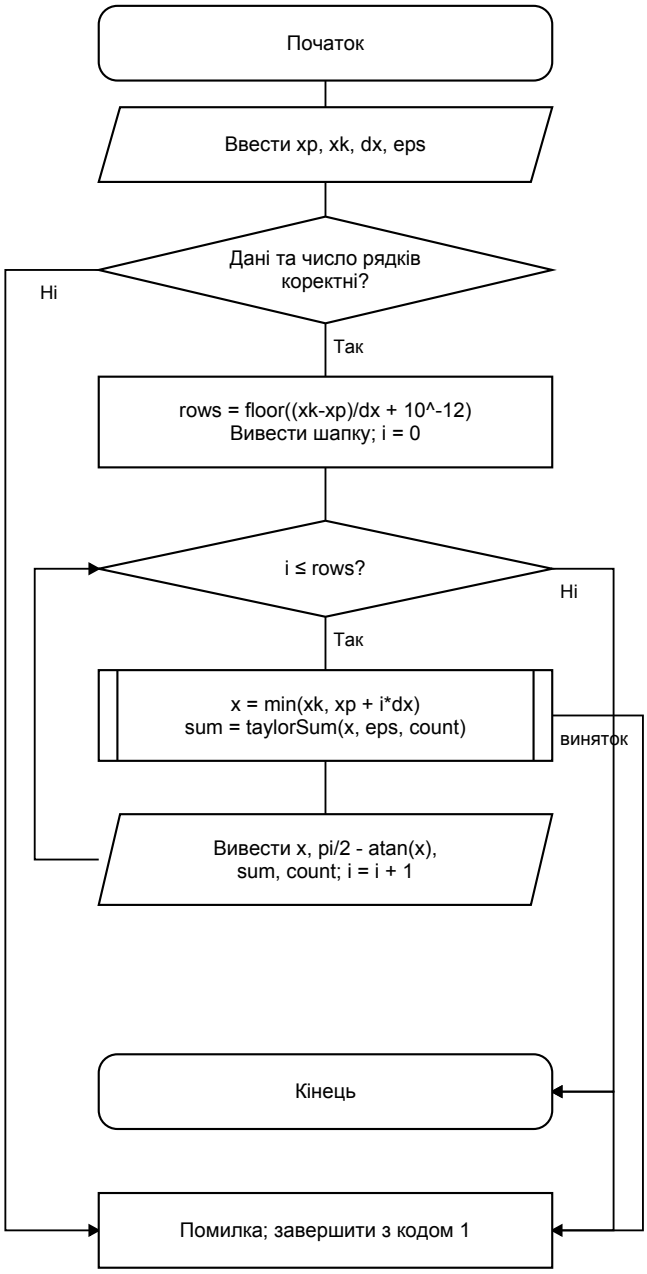
main	taylorSum → nextTerm

Git-репозиторій: _____

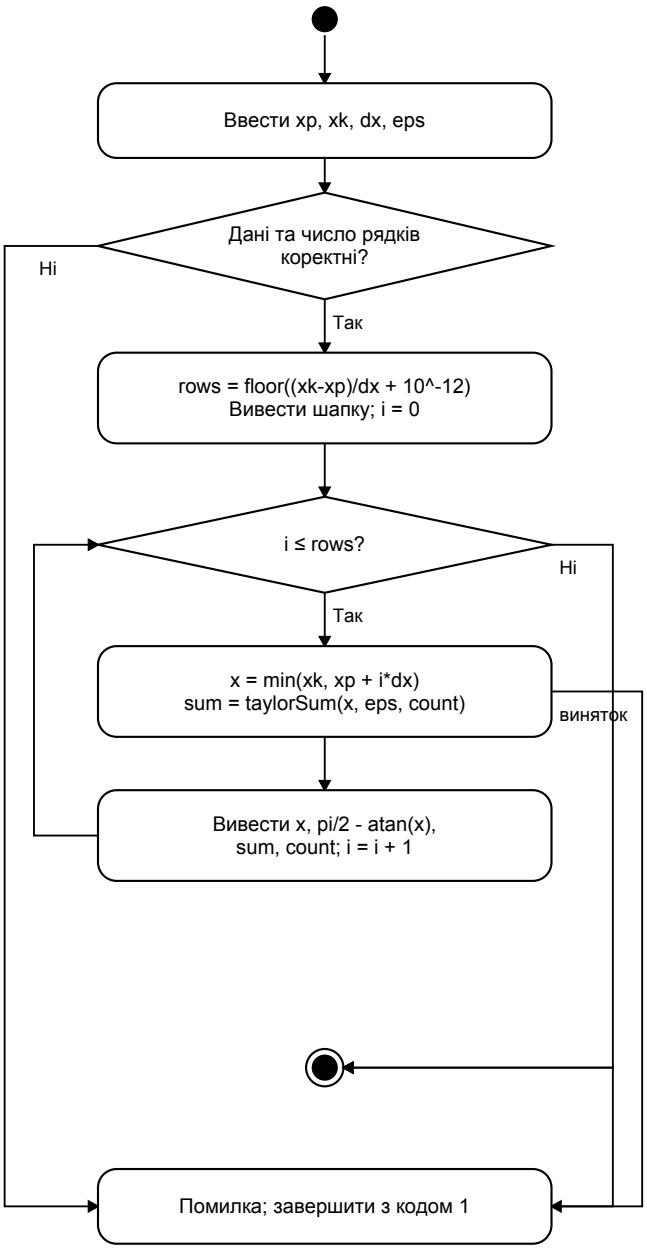
Алгоритм основної програми

main()

Блок-схема



UML-діаграма діяльності



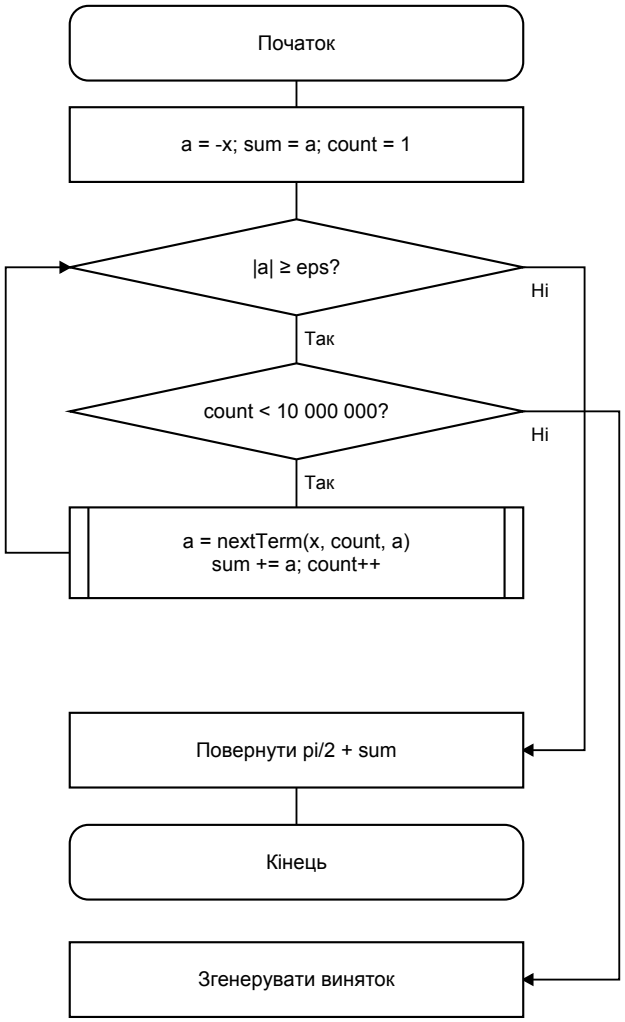
Введення: -1 ≤ xp ≤ xk ≤ 1, dx > 0, eps > 0; усі числа скінченні.

xk включається, якщо лежить на сітці з кроком dx.

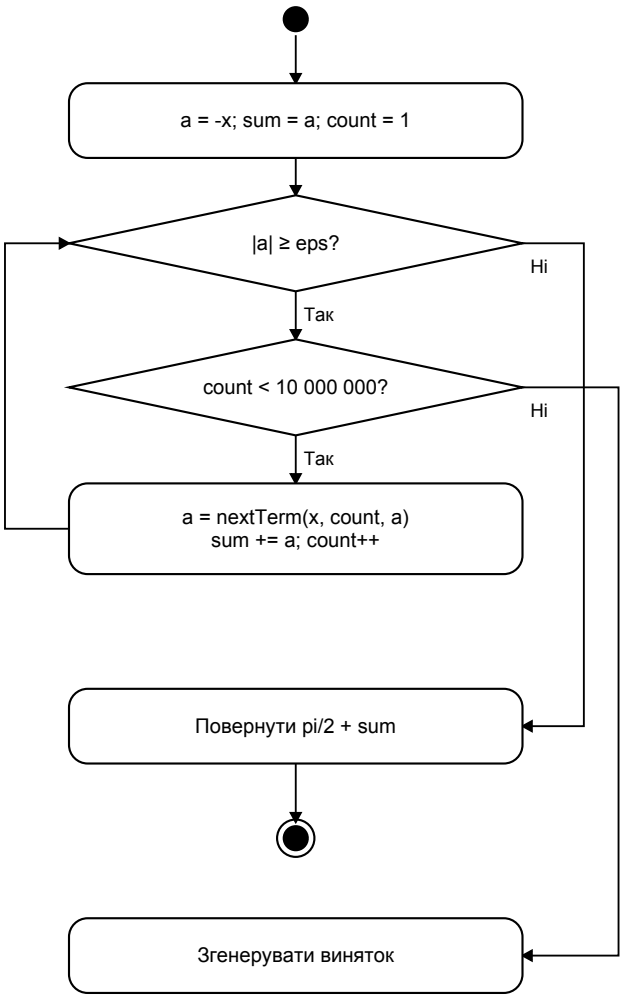
Алгоритми допоміжних функцій

taylorSum(x, eps, count)

Блок-схема

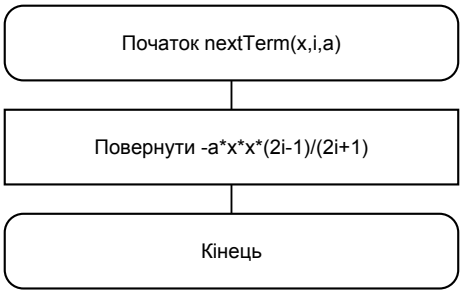


UML-діаграма діяльності

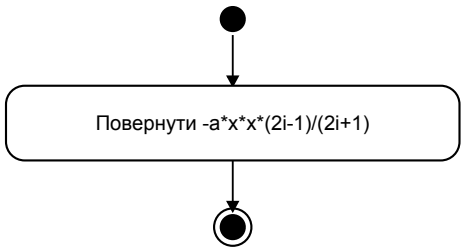


nextTerm(x, i, a)

Блок-схема



UML-діаграма діяльності



Текст програми та unit-тести

lab_5_2.cpp

```
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>
using namespace std;

// a_i = a_(i-1) * [-x^2 * (2i-1)/(2i+1)], i >= 1.
double nextTerm(double x, int i, double a)
{
    return -a * x * x * (2.0 * i - 1) / (2.0 * i + 1);
}

// Count includes a_0 = -x; pi/2 is not a series term.
double taylorSum(double x, double eps, int& count)
{
    double a = -x;
    double sum = a;
    count = 1;
    while (abs(a) >= eps)
    {
        if (count == 10000000)
            throw runtime_error("Too many terms; increase eps.");
        a = nextTerm(x, count, a);
        sum += a;
        ++count;
    }
    return acos(-1.0) / 2 + sum;
}

#ifndef UNIT_TEST
int main()
{
    double xp, xk, dx, eps;
    cout << "x_begin x_end dx eps: ";
    if (!(cin >> xp >> xk >> dx >> eps) ||
        !isfinite(xp) || !isfinite(xk) ||
        !isfinite(dx) || !isfinite(eps) ||
        xp < -1 || xk > 1 || xp > xk || dx <= 0 || eps <= 0)
        return 1;
    const double steps = (xk - xp) / dx;
    if (!isfinite(steps) || steps > 1000000)
    {
        cerr << "Too many rows; increase dx.\n";
        return 1;
    }
    const int rows = static_cast<int>(floor(steps + 1e-12));
    cout << "Lab 5.2 / Variant 7\n" << setw(12) << "x"
         << setw(16) << "arcctg(x)" << setw(16) << "sum"
         << setw(12) << "terms" << '\n';
    try
    {
        for (int i = 0; i <= rows; ++i)
        {
            const double x = min(xk, xp + i * dx);
            int count;
            const double sum = taylorSum(x, eps, count);
            const double exact = acos(-1.0) / 2 - atan(x);
            cout << fixed << setprecision(8) << setw(12) << x
```

Текст програми та unit-тести

```
                << setw(16) << exact << setw(16) << sum
                << setw(12) << count << '\n';
        }
    }
    catch (const exception& e)
    {
        cerr << e.what() << '\n';
        return 1;
    }
    return 0;
}
#endif
```

test_5_2.cpp

```
#define UNIT_TEST
#include "lab_5_2.cpp"

int main()
{
    int failed = 0;
    const double xs[] = {-1, -0.5, 0, 0.5, 1};
    for (double x : xs)
    {
        const double eps = abs(x) == 1 ? 1e-5 : 1e-10;
        int count;
        const double actual = taylorSum(x, eps, count);
        const double expected = acos(-1.0) / 2 - atan(x);
        const bool ok = abs(actual - expected) < eps + 1e-14
                        && count >= 1 && (x != 0 || count == 1);
        cout << (ok ? "PASS" : "FAIL") << " x=" << x
             << " terms=" << count << '\n';
        if (!ok) ++failed;
    }
    const bool termOk = abs(nextTerm(0.5, 1, -0.5)
                            - 1.0 / 24) < 1e-15;
    cout << (termOk ? "PASS" : "FAIL") << " nextTerm\n";
    if (!termOk) ++failed;
    cout << "5.2: " << 6 - failed << "/6 passed\n";
    return failed ? 1 : 0;
}
```

Результати перевірки та висновок

Фактичний вивід unit-тесту

```
PASS x=-1 terms=50001
PASS x=-0.5 terms=15
PASS x=0 terms=1
PASS x=0.5 terms=15
PASS x=1 terms=50001
PASS nextTerm
5.2: 6/6 passed
```

Приклад запуску

```
x_begin x_end dx eps: Lab 5.2 / Variant 7
           x       arcctg(x)             sum       terms
 -1.00000000      2.35619449      2.35619949       50001
 -0.50000000      2.03444394      2.03444560           7
  0.00000000      1.57079633      1.57079633           1
  0.50000000      1.10714872      1.10714705           7
  1.00000000      0.78539816      0.78539316       50001
```

Введення для прикладу: -1 1 0.5 0.00001.

На |x| = 1 збіжність повільна, тому кількість доданків більша.

Висновок: суму і доданок реалізовано окремо; усі 6 тестів пройдено.